

MARST Novelty Roadmap

A prioritised list of moves that raise MARST's novelty bar above "sensible engineering combination" without overclaiming. Each item names the contribution, the concrete steps, the files to touch, the effort estimate, and the specific reviewer objection it addresses.

The current state (commit `2524a41`): MARST beats 18 baselines on 4 traffic datasets at Wilcoxon $p < 0.001$ with strict causality and a leak audit. Documents `REPORT.md`, `WALKTHROUGH.md`, and `RELATED_WORK.md` position the work. The gap a strong reviewer will find: the *components* are textbook (LOCF/HA/KNN); the *combination* is well-precedented in spirit (ES-RNN, Wide & Deep, N-BEATS). The items below close that gap.

Tier 1 — biggest novelty levers

Pick at least **two** of these. Each one promotes MARST from "the LOCF+HA+KNN paper" to something more clearly novel.

A. Multi-scale anchors

Contribution. Promote each anchor family to a scale pyramid; let the softmax pick the right scale per (sensor, time). This is an architectural innovation not present in any cited prior work.

Concrete design.

```
LOCF family : {LOCF_short, LOCF_med, LOCF_long}      # 3 different soft-LOCF decays
HA family   : {HA_hourly, HA_daily, HA_weekly}      # 3 different periodicities
KNN family  : {KNN_1hop, KNN_2hop, KNN_3hop}        # 3 different spatial radii
```

The softmax gate now selects over **9 anchors** instead of 3, with a structured prior (group-lasso or hierarchical softmax) that encourages "one scale per family." Output:

```
 $\pi \in \Delta^9$                                 # gate over 9 anchors
blended =  $\sum_i \pi_i \cdot \text{anchor}_i$ 
prediction = blended +  $\alpha \cdot \text{residual}$ 
```

Files to touch. - `marst-multigpu.ipynb` cell 13: model class

`MaskedSTTransformerMARST.__init__` (add scale-bank parameters), `forward` (compute 9 anchors, project gate to 9-dim), `_compute_causal_signals` (return scale-banks). - `marst-multigpu.ipynb` cell 13: anchor ablation block — sweep individual scales and full families. - `REPORT.md` and `RELATED_WORK.md`: update the "what makes MARST novel" sections.

Effort. 2–3 days (1 day implementation + 1 day cross-dataset run + 1 day ablation).

Reviewer objection addressed. "Three classical anchors is a small toy." Answer: the architecture generalises to a scale-aware mixture; the 3-anchor version in the original paper is the base case of a structured family.

Risk. None significant. If a scale doesn't help, the gate learns to zero it out. Keep `K=9` as default and report the original `K=3` as an ablation.

B. Learnable / parametric anchors

Contribution. Replace hand-coded LOCF/HA/KNN with parametric anchor functions whose hyperparameters are learned end-to-end. Lifts the contribution from "we feed in textbook predictions" to "we propose a parametric family of causal imputation priors."

Concrete design.

- **LOCF** → `nn.Parameter` for `soft_locf_decay` per sensor (you currently have a single scalar 0.95). Initialise to 0.95; let it learn.
- **HA** → replace lookup of `ha_prior[t][n]` with $\sum_{k=1..K} a_k(n) \cdot \cos(2\pi kt/T) + b_k(n) \cdot \sin(2\pi kt/T)$ — a learned per-sensor Fourier expansion (like N-BEATS seasonality blocks). Pick $K = 4$ or 8 harmonics. Initialise to fit the empirical HA prior; then fine-tune.
- **KNN** → replace uniform $(adj \cdot x_{observed}) / (adj \cdot mask)$ with attention-style weighting: $\text{softmax}(\text{score}(\text{node_emb_i}, \text{node_emb_j})) \cdot x_j$ masked by adjacency. Lets the model learn which neighbours matter most.

Files to touch. - `marst-multigpu.ipynb` cell 13: model class — add `self.locf_decay = nn.Parameter(...)`, `self.ha_fourier_coeffs = nn.Parameter(...)`, `self.knn_attn = nn.Linear(...)`. Update `_compute_causal_signals` and `_knn_anchor`. - Add a separate "anchor identifiability" sub-section to the ablation: train MARST from scratch vs from HA-initialised Fourier coefficients; check whether the learned coefficients match the empirical HA prior.

Effort. 3–5 days (2 days implementation + 1 day debugging convergence + 2 days for cross-dataset and ablation).

Reviewer objection addressed. "LOCF/HA/KNN are decades old." Answer: those are the **initialisations** of our learnable anchor family; the converged forms are dataset-specific and qualitatively different (e.g. learned `decay` ranges from 0.82 on METR-LA to 0.97 on PEMS-BAY).

Risk. Convergence — learnable anchors can be unstable. Mitigation: regularise the parametric forms toward their classical counterparts for the first 200 epochs (`L2(decay - 0.95)`, `L2(fourier_coeffs - ha_init)`). Anneal regularisation away over training.

C. Cross-domain validation (the single biggest move)

Contribution. Apply MARST to non-traffic data. If it wins there too, the paper category changes from "a traffic imputation paper" to "a general spatiotemporal imputation paper." This roughly doubles the prestige tier of venues you can target.

Datasets to add.

1. **UCI Electricity** — 321 customer load curves at 15-min intervals. Has strong weekly + daily seasonality. URL: <https://archive.ics.uci.edu/ml/datasets/ElectricityLoadDiagrams20112014>. No native adjacency — construct one from load-pattern correlations or treat as fully-connected with attention.

2. **Beijing PM2.5 air quality** — 12 monitoring stations, hourly readings, geographic adjacency. URL: <https://archive.ics.uci.edu/ml/datasets/Beijing+PM2.5+Data>.
3. (Optional, stretch) **Solar production / NREL** or **Smart-meter dataset** for a third domain.

Expected story. Anchor mix π will be qualitatively different: - Electricity: HA dominates (daily/weekly cycles strong, LOCF less useful for spiky loads) - Air quality: KNN dominates (pollution is spatially diffusive) - Traffic: LOCF dominates on PEMS-BAY, HA elsewhere (already shown)

That story — "MARST adapts its anchor mix to the dominant regularity of the dataset" — is itself a publishable interpretability finding.

Files to touch. - `marst-multigpu.ipynb` cell 3: add new entries to `DATASETS` dict for `ELECTRICITY` and `AQI`. Provide URLs. - `marst-multigpu.ipynb` cell 13: `load_raw_array` and `load_adjacency` — add branches for the new dataset kinds (load curves and air quality). The non-traffic datasets may need synthetic adjacency (correlation-based or fully connected). - `marst-multigpu.ipynb` cell 13: regime edges and clamp ranges per dataset. - `marst-multigpu.ipynb` cell 15: add `'ELECTRICITY'` and `'AQI'` to `DATASETS_TO_RUN`. - New aggregation cell: cross-domain π comparison plot.

Effort. 3–7 days. Most time is in dataset preprocessing and finding the right adjacency for each. Training itself is fast on the GPU you have.

Reviewer objection addressed. "This is a domain-specific paper." Answer: identical model, identical hyperparameters (except anchor K), winning on three structurally different domains.

Risk. MARST may *not* dominate outside traffic — if HA is overwhelmingly best on electricity, MARST may match HA at best. Hedge: select cross-domain datasets where you've prototyped first (run a quick sanity check on each before committing the section).

Tier 2 — solid additions

Pick **one or two** depending on time budget.

D. Uncertainty-aware gating

Contribution. Replace plain softmax over anchors with **inverse-variance weighted** gating, giving free uncertainty estimates. Cite MoGU (2025), which made this move for forecasting.

Concrete design. Each anchor i produces (μ_i, σ_i^2) . Gate: $\pi_i \propto \exp(\text{logit}_i) / \sigma_i^2$. Output: $\mu = \sum_i \pi_i \mu_i$, $\sigma^2 = \sum_i \pi_i (\sigma_i^2 + (\mu_i - \mu)^2)$ (mixture variance). Train with NLL of Gaussian-mixture rather than MAE.

Files. Model class `forward`, loss in `_train_st`.

Effort. 2–3 days.

Reviewer concern addressed. "The model doesn't quantify its uncertainty." Adds a publishable side-deliverable that's useful for downstream traffic-management consumers.

Risk. Calibration. May need temperature scaling on σ^2 .

E. Online / streaming deployment protocol

Contribution. Demonstrate that MARST runs as a true streaming system — process one timestep at a time, update internal state, return predictions in <50ms on a CPU. Most published imputation methods are bidirectional and *cannot* run live.

Concrete deliverable. A new notebook section (or a small script) that: 1. Loads the trained MARST. 2. Iterates through the eval window one timestep at a time. 3. Maintains a sliding-window cache of the last `BATCH_TIME=48` observations. 4. Reports per-step latency (mean, p95, p99) on CPU and GPU.

Files. New cell at the end of `marst-multigpu.ipynb` (or a sibling script `streaming_demo.py`).

Effort. 1–2 days.

Reviewer concern addressed. "Is this actually deployable?" Strong differentiation against ImputeFormer / STAMImputer / GRIN (all bidirectional / offline).

Risk. None. Either the model meets the latency target or it doesn't; the negative result is also publishable as a limitations note.

F. Regime-Aware MAE (JamMAE) as a named metric

Contribution. Formalise JamMAE in the paper as a *named, citable evaluation protocol* for traffic imputation. Future papers will cite you for this.

Concrete steps. 1. Define regime edges precisely in the methods section (e.g. `q20` for jam in speed datasets, `q40` for free-flow). 2. Report JamMAE for *every* baseline you reproduce (you already do this — just promote it to a named contribution). 3. Add a 1-paragraph subsection "Regime-Aware Evaluation" in the paper that proposes JamMAE as a metric.

Files. `REPORT.md` and the paper draft. The numbers are already produced by the notebook.

Effort. <1 day.

Reviewer concern addressed. "Overall MAE rewards models that are mediocre everywhere." Adds an evaluation-methodology contribution at trivial cost.

Risk. None.

G. Compute-matched mini-MARST

Contribution. Kill the parameter-count objection (MARST 1.67M vs iTransformer 96K vs BRITS 14.5K).

Concrete steps. Train MARST with `hidden=64, n_layers=3` ($\approx 200K$ params) and report. If it still wins significantly against iTransformer, the comparison is fair.

Files. New cell after cell 15 in the notebook, or a sibling run with overridden hyperparameters. New row in the cross-dataset MAE table.

Effort. 1 day (one training run per dataset).

Reviewer concern addressed. "MARST wins because it's $17\times$ larger." Direct rebuttal with a compute-matched comparison.

Risk. Mini-MARST may not match the full model. Hedge: report both — full MARST for the headline, mini-MARST as the compute-matched comparison.

Tier 3 — depth and credibility

These don't add architectural novelty but raise the scientific rigour and protect against reviewer skepticism.

H. Formal analysis of the anchor mixture

Contribution. A short (~ 1 page) analytical section showing that, for any convex loss and any per-position softmax over K anchors, MARST's expected loss is bounded above by $\min_i E[L(\text{anchor}_i, \text{truth})]$ — i.e., the mixture is provably never worse than the best individual anchor in expectation. Brief proof; cite the convexity of the loss.

Files. Paper draft. Optionally a short appendix subsection.

Effort. 1 day.

Reviewer concern addressed. "This is purely empirical." Adds a citable lemma without claiming a major theorem.

Risk. None. The bound is standard convex analysis.

I. Failure-mode analysis

Contribution. A section titled "When does MARST fail?" with concrete failure cases. Reviewers reward this signal of scientific honesty.

Concrete cases to test. 1. **Very high sparsity ($>95\%$)** — does the mixture still work when individual anchors are unreliable? 2. **Adversarial outages** — whole neighbourhoods go dark (KNN fails) or whole hours go dark (HA fails). 3. **Distribution shift** — train on weekdays, test on weekends. Or train on PEMS-BAY, test on METR-LA without retraining. 4. **Wrong adjacency** — perturb the road graph (drop 30% of edges, add 30% of wrong edges) and measure degradation.

Files. Add a "Failure modes and limitations" cell after the main results in the notebook. Surface findings in `REPORT.md`.

Effort. 2–3 days.

Reviewer concern addressed. "This paper oversells." Adds a scientific-honesty section that elevates the manuscript.

Risk. May reveal genuine weaknesses — but discovering them privately now is better than during peer review.

J. Adversarial / wrong-adjacency robustness

Contribution. Show MARST degrades gracefully when the road graph is wrong, because LOCF + HA anchors don't depend on the graph (only KNN does). This is a unique advantage of multi-anchor decomposition that competing graph-based methods (GRIN, DCRNN, GWN) cannot claim.

Concrete experiment. At test time: 1. Drop X% of true edges ($X \in \{10, 30, 50, 70, 90\}$) 2. Add X% of random false edges 3. Measure MARST MAE, GRIN MAE, DCRNN MAE

Expected result: GRIN/DCRNN collapse; MARST stays close to its clean number because LOCF + HA still work.

Files. New cell after the leak audit. New plot.

Effort. 2 days.

Reviewer concern addressed. None directly — but it's a strong positive: a unique selling point of the anchor decomposition.

Risk. None.

Anti-roadmap: what NOT to do

These have negative expected value and should be avoided unless you have a specific reason.

- **Don't propose a new transformer architecture.** You'd be competing on the wrong axis. The next iTransformer/PatchTST will dethrone yours within a year. Stay focused on the decomposition story.
 - **Don't add more 2024–2026 baselines for the sake of it.** You already beat 18. Adding 5 more risks marginal-improvement reviews and dates the paper faster.
 - **Don't add diffusion / generative refinement.** Crowded space; you'd lose the causality differentiator (most diffusion imputation papers are offline).
 - **Don't make the model bigger.** The parameter-count objection gets worse, not better.
 - **Don't write a "system" paper unless you do E (streaming).** Half-deployment is worse than no claim.
 - **Don't pursue federated / multi-region MARST.** Tangential to the core contribution; entire separate paper.
-

Suggested execution plans

One-week plan (minimum viable upgrade)

- **A** (multi-scale anchors) — 2 days
- **I** (failure modes) — 2 days
- **G** (compute-matched) — 1 day
- **F** (name JamMAE) — 0.5 day

Outcome: paper with an architectural contribution (A), scientific honesty (I), fair parameter comparison (G), and a named evaluation metric (F). Ready for a workshop or low-tier venue.

Two-to-three-week plan (target a mid-tier venue)

Add to the one-week plan: - **B** (learnable anchors) — 4 days - **E** (streaming demo) — 2 days - **H** (formal analysis) — 1 day

Outcome: architectural novelty + theoretical depth + deployment story. Suitable for AAAI / IJCAI / SIGKDD workshop tracks, or a strong arXiv preprint.

One-month-plus plan (target a top venue)

Add to the two-week plan: - **C** (cross-domain on Electricity + AQI) — 7 days - **J** (adjacency robustness) — 2 days - **D** (uncertainty gating) — 3 days

Outcome: a general-method paper with cross-domain validation, formal guarantees, uncertainty quantification, deployment protocol, and robustness analysis. Target ICLR / NeurIPS / KDD main track.

Decision matrix

Item	Novelty gain	Effort (days)	Risk	Tier-1 venue impact
A. Multi-scale anchors	High	2–3	Low	Yes
B. Learnable anchors	High	3–5	Medium	Yes
C. Cross-domain	Highest	3–7	Medium	Yes
D. Uncertainty gating	Medium	2–3	Low	Partial
E. Streaming deployment	Medium	1–2	None	Yes
F. Name JamMAE	Low	<1	None	Marginal
G. Compute-matched	Low	1	None	Defensive
H. Formal analysis	Medium	1	None	Yes
I. Failure modes	Medium	2–3	None	Yes
J. Adjacency robustness	Medium	2	None	Partial

Implementation checkpoint: what to do first

If you can do exactly one thing this week, do **A (multi-scale anchors)**. It's the lowest-risk move that: 1. Directly answers the "what's new architecturally?" question 2. Strengthens the anchor ablation (you can now ablate by family AND by scale) 3. Improves quantitative results (more anchors → better fit) 4. Costs only 2–3 days

If you can do two things, add **G (compute-matched)** — it's a 1-day defensive move that closes the biggest fair-comparison criticism.

If you have a free third day, do **F (name JamMAE)** — it's almost free and seeds future citations.

After that, the next decision is whether to pursue depth (B + H + I) or breadth (C). For a top-tier paper submission, breadth (C) wins; for a stronger but narrower paper, depth wins. They're not mutually exclusive on a longer timeline.